

Министерство образования Республики Беларусь

Учреждение образования

БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ИНФОРМАТИКИ
И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра информатики

Дисциплина Архитектура вычислительных систем

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к курсовому проекту
на тему

**СРАВНЕНИЕ ПРОИЗВОДИТЕЛЬНОСТИ ПРОЦЕССОРОВ INTEL
CORE I7-12650H И RYZEN 7 8845HS НА ОСНОВЕ АЛГОРИТМА
ПУЗЫРЬКОВОЙ СОРТИРОВКИ**

БГУИР КП 6-05-0612-02 017 ПЗ

Студент

Е. Д. Мартынкевич

Руководитель

А. А. Калиновская

Нормоконтролёр

А. А. Калиновская

Минск 2025

СОДЕРЖАНИЕ

Введение.....	4
1 Архитектура вычислительной системы.....	5
1.1 Структура и архитектура вычислительной системы.....	5
1.2 История, версии и достоинства.....	6
1.3 Обоснование выбора вычислительной системы.....	7
1.4 Анализ выбранной вычислительной системы для написания программы.....	8
2 Платформа программного обеспечения.....	9
2.1 Структура и архитектура платформы.....	9
2.2 История, версии и достоинства.....	11
2.3 Обоснование выбора платформы.....	12
2.4 Анализ операционной системы для написания программы.....	13
3 Теоретическое обоснование разработки программного продукта.....	14
3.1 Обоснование необходимости разработки.....	14
3.2 Технологии программирования, используемые для решения поставленных задач.....	15
3.3 Связь архитектуры вычислительной системы с разрабатываемым программным обеспечением.....	16
4 Проектирование функциональных возможностей программы.....	18
4.1 Общая цель программы.....	18
4.2 Теоретические сведения о пузырьковой сортировке.....	18
4.3 Описание функциональных возможностей.....	19
5 Сравнение производительности процессоров.....	21
5.1 Общая структура программы.....	21
5.2 Описание функциональной схемы программы.....	22
5.3 Описание блок схемы алгоритма программы.....	23
5.4 Подготовка к тестированию.....	24
5.5 Результаты измерений с включенным Hyper-Threading/SMT.....	25
5.6 Результаты измерений с выключенным Hyper-Threading/SMT.....	30
Заключение.....	42
Список литературных источников.....	43
Приложение А (обязательное) Справка о проверке на заимствования.....	45
Приложение Б (обязательное) Листинг программного кода.....	46
Приложение В (обязательное) Функциональная схема алгоритма, реализующего программное средство.....	50
Приложение Г (обязательное) Блок схема алгоритма, реализующего программное средство.....	51
Приложение Д (обязательное) Графики сравнений производительности процессоров.....	52
Приложение Е (обязательное) Графический интерфейс пользователя.....	53
Приложение Ж (обязательное) Ведомость курсового проекта.....	54

ВВЕДЕНИЕ

В современном мире информационных технологий процессоры играют ключевую роль в обеспечении вычислительной мощности компьютерных систем. Эффективность работы процессора имеет большое значение как для повседневных пользователей, так и для профессиональных разработчиков. С ростом числа ядер и потоков в современных процессорах возрастает и потребность в анализе их производительности при выполнении различных алгоритмических задач. Одним из таких алгоритмов является пузырьковая сортировка, которая, несмотря на свою простоту, часто используется в учебных целях для оценки вычислительных ресурсов и сравнения эффективности различных архитектур.

В рамках данного проекта проводится сравнительный анализ производительности двух современных процессоров – *Intel Core i7-12650H* и *AMD Ryzen 7 8845HS* – при выполнении алгоритма пузырьковой сортировки. Оба процессора применяются в современных ноутбуках и игровых системах, отличаются архитектурными особенностями, что делает их интересными объектами для исследования с точки зрения обработки данных.

Актуальность темы курсового проекта обусловлена растущей потребностью в эффективных вычислительных ресурсах для решения сложных задач в различных областях, таких как наука, инженерия и информационные технологии.

Цель проекта заключается в проведении сравнительного анализа производительности процессоров *Intel Core i7-12650H* и *AMD Ryzen 7 8845HS* при выполнении алгоритма пузырьковой сортировки. Для достижения этой цели в ходе исследования будут решены следующие задачи:

- 1 Изучение технических характеристик процессоров, а также их архитектурных особенностей.

- 2 Разработка программы для реализации алгоритма пузырьковой сортировки на языке программирования C++.

- 3 Проведение вычислительных экспериментов с различным числом активных ядер.

- 4 Анализ полученных данных для оценки эффективности процессоров.

В результате исследования будут построены графики, демонстрирующие время выполнения сортировки массивов различного размера, что позволит провести детальный анализ производительности рассматриваемых процессоров.

Данный курсовой проект выполнен мной лично, проверен на заимствования, процент оригинальности составляет 77% (отчет о проверке на заимствования прилагается).

1 АРХИТЕКТУРА ВЫЧИСЛИТЕЛЬНОЙ СИСТЕМЫ

1.1 Структура и архитектура вычислительной системы

В рассматриваемой работе для сравнительного анализа производительности выбраны два современных мобильных процессора — *Intel Core i7-12650H* и *AMD Ryzen 7 8845HS*. Оба решения ориентированы в первую очередь на ноутбуки и мобильные рабочие станции, но представляют разные архитектурные подходы: Intel использует гибридную архитектуру с «Performance» и «Efficient» ядрами, тогда как AMD опирается на традиционную многопоточную конфигурацию с высокими тактовыми частотами и современным техпроцессом.

Основные характеристики выбранных процессоров представлены в таблице 1.1 [1].

Таблица 1.1 – Основные характеристики выбранных процессоров

Характеристики	<i>Intel Core i7-12650H</i>	<i>AMD Ryzen 7 8845HS</i>
Кодовое имя архитектуры	<i>Alder Lake H</i>	<i>Hawk Point / Zen 4</i>
Физические ядра	10	8
Количество потоков	16	16
<i>L1</i> кэш	800 KB	512 KB
<i>L2</i> кэш	12.5 MB	8 MB
<i>L3</i> кэш	24 MB	16 MB
Тактовая частота	2.3 - 4.7 GHz	3.8 - 5.1 GHz
Техпроцесс	10 nm	4 nm
Разрядность	64 bit	64 bit
Встроенная графика	<i>Intel UHD Graphics</i>	<i>AMD Radeon 780M</i>
Тип памяти	<i>DDR4, DDR5</i>	<i>DDR5</i>
Архитектура	<i>x86-64</i>	<i>x86-64</i>
<i>Hyper-Threading/SMT</i>	+	+
Теплопакет (<i>TDP</i>)	45 Вт	45 Вт

Intel Core i7-12650H – мобильный процессор 12-го поколения с гибридной архитектурой (комбинация *Performance* и *Efficient* ядер). В конфигурации он содержит 10 физических ядер (6 Р-ядра + 4 Е-ядра) и поддерживает до 16 потоков выполнения благодаря *Hyper-Threading* на Р-ядрах; частоты Р-ядер варьируются в широком диапазоне и в пике могут достигать ~4.7 GHz. Процессор производится по техпроцессу *Intel 7* (Enhanced 10 нм) и оснащён крупным уровнем совместного L3-кэша (*Intel Smart Cache*), а типичное энергопотребление для мобильной версии — около 45 Вт. Встроенная графика *Intel Iris Xe* обеспечивает достаточную производительность в мультимедийных сценариях и служит вспомогательным блоком при тестировании плат. Эти характеристики и гибридная организация

ядер определяют баланс между энергопотреблением и пиковым быстродействием в задачах разной параллельности.

AMD Ryzen 7 8845HS мобильный процессор семейства *Ryzen 8000 (Hawk Point / Zen 4)*, ориентированный на производительные ноутбуки класса HS. Он имеет 8 физических ядер с поддержкой SMT, что даёт 16 потоков, и обладает высокой турбо-частотой до 5.1 GHz при базовой частоте порядка 3.8 GHz. Процессор использует современный техпроцесс и рассчитан на TDP порядка 45 Вт в классе HS, располагает значительным объёмом кэш-памяти L2/L3 и интегрированной графикой *Radeon 780M* (RDNA архитектура), обеспечивающей хорошую графическую производительность без дискретной карты. Высокие одноядерные частоты и архитектурные оптимизации Zen 4 делают эту модель сильной в задачах с выраженной последовательной частью и в сценариях, чувствительных к частоте ядра [3].

Итак, *Intel Core i7-12650H* обеспечивает сбалансированную производительность благодаря гибридной архитектуре и возможностям энергоменеджмента, тогда как *AMD Ryzen 7 8845HS* ориентирован на высокую одноядерную частоту и сильную «частотную» производительность — что важно учитывать при сравнении на базе однопоточного теста, такого как пузырьковая сортировка.

1.2 История, версии и достоинства

Intel Core i7-12650H и *AMD Ryzen 7 8845HS* — это заметные представители своих семейств мобильных процессоров, каждый из которых имеет собственную историю развития, набор версий и ряд достоинств, важных для практических вычислений.

Intel Core i7-12650H был анонсирован и выпущен в начале 2022 года как часть 12-го поколения (*Alder Lake*) мобильных процессоров. Эта модель сочетает в себе 10 физических ядер (6 Р-ядер + 4 Е-ядра) и до 16 потоков, оснащена объединённым L3-кэшем объёмом порядка 24 МБ и производится по техпроцессу *Intel 7*. Конфигурация с гибридными кластерами даёт *i7-12650H* преимущества в задачах с переменным характером нагрузки: Р-ядра обеспечивают высокую производительность для тяжёлых участков кода, тогда как Е-ядра повышают энергоэффективность при фоновом или параллельном выполнении мелких задач. Типичное тепловыделение для мобильной реализации — около 45 Вт (настраиваемый TDP в зависимости от платформы).

AMD Ryzen 7 8845HS относится к семейству *Ryzen 8000* и был выпущен в 2023 году как модель класса HS для производительных ноутбуков. Процессор имеет 8 физических ядер с поддержкой SMT (в итоге 16 потоков), высокие пиковые частоты (вплоть до ~5,1 GHz) и современную топологию, ориентированную на сочетание одно- и многопоточной эффективности. Конструктивно серия HS нацелена на сохранение высокого уровня производительности при ограничениях по энергопотреблению (типичный TDP порядка 45 Вт для HS-конфигураций), а интегрированная графика *Radeon*

780M обеспечивает дополнительные возможности в мультимедийных сценариях без дискретного GPU.

Оба процессора являются результатом эволюции архитектур своих производителей и сочетают множество инженерных решений, направленных на повышение производительности и энергоэффективности. Для задач тестирования вычислительных ресурсов — и, в частности, для сравнительного анализа на основе алгоритма пузырьковой сортировки — важны следующие достоинства: у *Intel*-модели — гибридная организация ядер и развитые механизмы управления энергопотреблением и потоками, у *AMD*-модели — высокая одноядерная частота и оптимизации микроархитектуры, которые дают преимущество в последовательных или кэше-чувствительных алгоритмах. Эти особенности определяют методологию и интерпретацию экспериментов в данной работе.

1.3 Обоснование выбора вычислительной системы

Выбор процессоров *Intel Core i7-12650H* и *AMD Ryzen 7 8845HS* для сравнительного анализа на основе алгоритма пузырьковой сортировки обосновывается следующими факторами:

1 Архитектурные различия и многозадачность — оба процессора имеют многоядерную организацию и поддержку многопоточности (гибридная конфигурация P- и E-ядер у *Intel Core i7-12650H* и традиционная многоядерная компоновка у *Ryzen 7 8845HS*). Эти особенности определяют характер распределения вычислительной нагрузки и влияют на поведение как параллельных, так и последовательных алгоритмов, что важно при сравнении производительности на эталонном тесте.

2 Технологии повышения производительности — *Intel* и *AMD* применяют разные подходы к ускорению вычислений: у *Intel* — гибридный дизайн и механизмы управления ядрами/планировщик, у *AMD* — высокая одноядерная частота и оптимизации микроархитектуры с поддержкой *SMT*. Пузырьковая сортировка в типичной реализации преимущественно последовательна и чувствительна к одноядерной производительности и организации кэша, поэтому сочетание этих технологий позволит выявить их влияние на практическое время выполнения.

3 Репрезентативность и актуальность — оба процессора широко используются в современных производительных ноутбуках и мобильных рабочих станциях, что делает их сравнение релевантным для пользователей и исследователей, интересующихся реальными сценариями обработки данных и влиянием архитектуры на простые алгоритмы сортировки.

4 Доступность и управляемость условий эксперимента — наличие этих платформ и возможности настройки параметров (например, TDP, частотных профилей и числа активных ядер/потоков) позволяют провести воспроизводимые вычислительные эксперименты. Это обеспечит корректный сбор данных для анализа и последующего построения графиков времени

выполнения пузырьковой сортировки при разных размерах входных массивов и конфигурациях процессора.

Таким образом, выбор *Intel Core i7-12650H* и *AMD Ryzen 7 8845HS* оправдан их различной архитектурой, поддержкой технологий повышения производительности, широкой практической применимостью и возможностью настройки экспериментальных условий — всё это позволит получить надежные и интерпретируемые результаты при сравнении вычислительной эффективности на основе алгоритма пузырьковой сортировки.

1.4 Анализ выбранной вычислительной системы для написания программы

Для анализа производительности выбранных процессоров *Intel Core i7-12650H* и *AMD Ryzen 7 8845HS* в контексте реализации и тестирования алгоритма пузырьковой сортировки использовались результаты типичных синтетических бенчмарков: *Cinebench R23*, *GeekBench v6* и *PassMark*. Эти наборы тестов позволяют получить представление о поведении процессоров как в однопоточном, так и в многопоточном режимах, а также о их способности справляться с разнохарактерными вычислительными нагрузками, что важно при оценке времени выполнения алгоритмических задач.

Cinebench – это инструмент для бенчмаркинга, который в основном используется для стресс-тестирования вашего ЦП и сравнения его с другими процессорами [4].

Geekbench – это кроссплатформенный тест производительности системы (бенчмарк). *Geekbench 6* включает обновленные рабочие нагрузки ЦП, которые моделируют реальные задачи и приложения. *Geekbench* – это бенчмарк, который отражает то, с чем сталкиваются реальные пользователи на своих мобильных устройствах и персональных компьютерах [5]. Тесты охватывают множество категорий, включая вычисления с плавающей точкой, криптографию и многозадачность, что делает его полезным для оценки универсальной производительности системы.

PassMark – выполняет сложные математические вычисления, которые нагружают производительность процессора при выполнении задач, связанных с вычислениями, сжатием, шифрованием и тому подобным [6].

2 ПЛАТФОРМА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

2.1 Структура и архитектура платформы

Для выполнения данного курсового проекта в качестве программной платформы была выбрана операционная система *Linux*. Далее будут рассмотрены основные характеристики, структура и архитектура ОС.

Операционная система *Linux* имеет модульную структуру. Это означает, что она состоит из четко разделяемых компонентов, каждый из которых отвечает строго за свои функции. То есть функциональность ОС распределена по модулям и подгружается постепенно, по мере необходимости. Так как все части ядра функционируют в едином адресном пространстве, его также называют монолитным. Однако в архитектуру могут входить и элементы микроядра [8]. Вот основные компоненты и архитектурные особенности платформы *Linux*:

1 Ядро *Linux* – это основной компонент, управляющий аппаратным обеспечением и взаимодействием с другими компонентами ОС. Задачи включают управление памятью, процессами и устройствами. Ядро *Linux* представляет собой микроядро, которое обеспечивает базовые функции, включая планирование задач, управление памятью и взаимодействие с устройствами. [9].

2 Системные библиотеки – это специальные программы, которые предоставляют интерфейс для доступа приложений к функциям ядра операционной системы. Они обрабатывают системные вызовы, передавая их от приложений к ядру для выполнения задач, таких как управление памятью, работа с файловыми системами, управление потоками и взаимодействие с аппаратными устройствами. Примерами системных библиотек *Linux* являются *GNU C Library (glibc)* для языка *C*, *libstdc++* для языка *C++* и *libpthread* для работы с потоками.

3 Утилиты и инструменты. Под системными утилитами понимают программы, отвечающие за выполнение отдельных специализированных задач управления. *Linux* поставляется с множеством утилит и инструментов командной строки, которые позволяют управлять системой, настраивать ее, администрировать и разрабатывать приложения. Примеры таких утилит: *bash*, *grep*, *sed*, *awk*, *gcc*, *make* и многие другие [10].

4 Файловая система. Файловая система отвечает за организацию и хранение данных на жестком диске или других устройствах хранения. *Linux* поддерживает различные файловые системы, например, *ext* различных версий, *Btrfs (B-Tree File System)*, *JFS*, *XFS*. *SWAP* [11].

5 Драйверы устройств. *Linux* имеет обширную поддержку для различного аппаратного обеспечения. Драйверы устройств позволяют операционной системе взаимодействовать с железом компьютера, таким как видеокарты, сетевые адаптеры, звуковые карты и другие устройства.

6 Модульность. *Linux* поддерживает модульность, что позволяет добавлять или удалять функциональность ядра динамически во время работы системы. Это позволяет оптимизировать использование ресурсов и обновлять ядро без перезагрузки системы.

На рисунке 2.1 представлена упрощенная структура операционной системы.

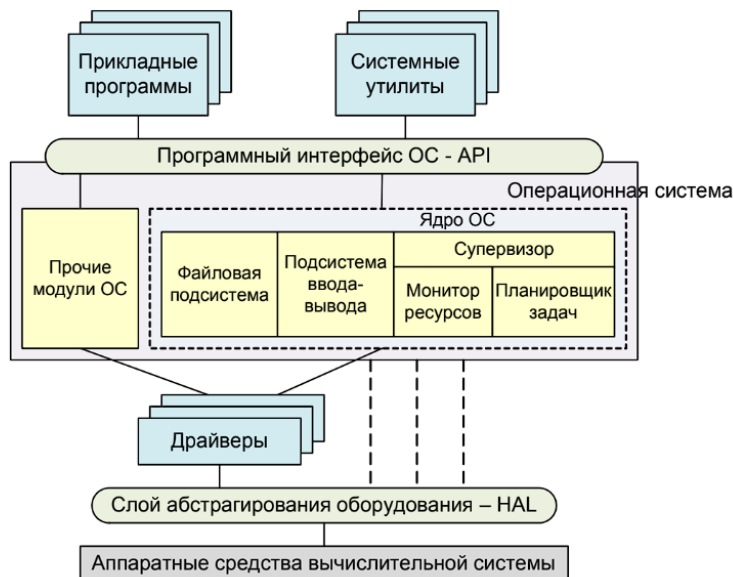


Рисунок 2.1 – Упрощенная структура ОС

Уровень абстрагирования от оборудования (*HAL – Hardware Abstraction Layer*) – изоляция вышестоящих уровней от технической платформы.

На рисунке 2.2 продемонстрирована традиционная архитектура *Unix*-системы (макроядро).



Рисунок 2.2 – Традиционная архитектура *Unix*-системы

2.2 История, версии и достоинства

Unix начал свое развитие в 1969 году, в стенах американской компании *Bell Labs*, где группа программистов заложила его основу. Чтобы сделать разработку *Unix* более независимой от конфигурации компьютера, Денис Ритчи и Кен Томпсон создали язык программирования Си. Этот язык по-прежнему используется для написания ключевых компонентов большинства операционных систем.

К середине 70-х годов *Unix* широко распространился, хотя его использование было ограничено крупными предприятиями и университетами, так как персональные компьютеры в те времена были редкостью. *Linux* распространялся бесплатно, и каждая организация могла адаптировать систему под свои нужды, в результате чего возникли множество вариаций *Unix*, включая коммерческие.

В начале 80-х годов *Unix* выбрали как основную платформу для развития сетевого протокола *TCP/IP*, что значительно поспособствовало её укреплению на серверном рынке.

Тем не менее, коммерческие версии *Unix* замедляли прогресс, так как фирмы, производившие свои дистрибутивы, не позволяли распространять исходные коды, что препятствовало другим разработчикам пользоваться уже написанным кодом.

В 1982 году Ричард Столлман столкнулся с ограничением лицензий ПО, мешавшим обмену исходниками в научных кругах. Покинув рабочее место, в 1983 году он инициировал проект *GNU*, целью которого стало создание системы с открытым исходным кодом.

В 1987 году Эндрю Таненбаум создал учебную *Unix*-подобную систему *Minix*. Она служила учащимся инструментом для освоения основ ОС и состава её компонентов, распространявшаяся по свободной лицензии для изучения исходного кода.

Ubuntu стал важным этапом для появления *Linux*, так как большинство студентов учились на её основе.

В начале 90-х, Линус Торвальдс, обучавшись на базе *Minix*, ощутил нехватку возможностей этой учебной системы и начал работу над собственной ОС, начиная с разработки системных вызовов. К лету 1991 года, он создал первую версию ядра, впоследствии получившего название *Linux*.

Важно отметить, что ядро операционной системы не является полной ОС с точки зрения пользователя. Оно служит для взаимодействия программ с оборудованием, управляет распределением памяти и процессорного времени.

Linux стал активно развиваться благодаря открытому доступу и поддержке сообщества программистов со всего мира.

Поскольку проекту *GNU* недоставало только ядра, роль *Linux* оказалась весьма кстати. Это ядро интегрировалось с ПО проекта *GNU*, формируя полноценную операционную систему, известную как *GNU/Linux*, а часто просто называемую *Linux*.

Если изначально *Linux* ориентировался на специалистов, то впоследствии дистрибутивы стали более доступными для обычных пользователей.

Дистрибутивы различаются не столько по графическому интерфейсу, сколько по области применения – для серверов или настольных компьютеров, а также по способу управления пакетами и другим функциям.

С течением времени *Linux* была адаптирована для различных архитектур и теперь используется в самых разнообразных электронных устройствах. Примечательно, что *Android* тоже построен на базе ядра *Linux*.

Тем не менее, на текущий момент системы *GNU/Linux* чаще всего применяются на серверах, включая веб-серверы и суперкомпьютеры, и в меньшей степени на настольных ПК [12].

Версии ядра *Linux* [13]:

Версия 1.x (1991 – 1995): версия ядра *Linux*, созданная Линусом Торвальдсом. Включала базовые функции, такие как управление процессами, файловой системой и базовое взаимодействие с оборудованием.

Версия 2.x (1996 – 2003): поддержка многопроцессорных систем и улучшенное управление памятью, улучшение сетевого стека и масштабируемости, поддержка новых файловых систем, таких как *ext3*.

Версия 3.x (2011 – 2013): объединение ряда нововведений, включая поддержку улучшенных драйверов, более быстрые файловые системы и новые механизмы планирования задач.

Версия 4.x (2015 – 2019): повышенная энергоэффективность, поддержка файловой системы *Btrfs*, развитие технологий сетей и драйверов.

Версия 5.x (2019 – 2022): улучшения для *ARM*, *RISC-V*, расширение функций безопасности, оптимизация файловых систем и производительности.

Версия 6.x (2022 – настоящее время): Оптимизация для серверов и встраиваемых систем, улучшенная поддержка современных аппаратных платформ и виртуализации.

2.3 Обоснование выбора платформы

В рамках данного курсового проекта было принято решение использовать *Linux Ubuntu* и этот выбор обоснован следующими факторами:

1 Стабильность и производительность: *Linux Ubuntu* зарекомендовал себя как стабильная и производительная система. Это позволяет минимизировать влияние сбоев и сторонних факторов на результаты экспериментов.

2 Легковесность и прозрачность: Операционная система более легковесна по сравнению с другими ОС и позволяет гибко управлять использованием системных ресурсов. Это помогает получить более точные и надежные результаты тестов.

3 Доступ к инструментам разработки: Операционная система включает широкий набор инструментов для разработки и отладки программ, таких как

GCC и *Clang*. Это упрощает работу с кодом на C++.

4 Основы на ядре *Linux*: *Linux Ubuntu*, основанный на ядре *Linux*, предоставляет возможность точной настройки системы для проведения вычислительных экспериментов. Это особенно важно для исследования производительности процессоров при решении систем линейных уравнений методом Гаусса.

Таким образом, выбор *Linux Ubuntu* был сделан на основе её стабильности, возможностей для настройки и поддержки многозадачности и многопоточности. Эти преимущества делают *Linux Ubuntu* оптимальной средой для выполнения целей исследования, связанных с анализом производительности многоядерных процессоров, и проведения вычислительных экспериментов.

2.4 Анализ операционной системы для написания программы

Linux Ubuntu – это один из самых популярных дистрибутивов *Linux*, ориентированный на удобство использования, стабильность и гибкость. Он идеально подходит для пользователей, которые привыкли к операционным системам *Windows* и ищут легкую, функциональную и доступную альтернативу. Дистрибутив *Linux Ubuntu* включает в себя множество встроенных инструментов и приложений, что делает его готовым к использованию сразу после установки. Это дает возможность пользователю не тратить время на настройку и сразу переходить к работе.

Также не секрет, что *Ubuntu* является наиболее популярным *Linux*-дистрибутивом, поэтому множество разработчиков создают свой софт, прежде всего, для *Ubuntu*, гарантируя совместимость и поддерживаемость мультимедийных и офисных программ, компьютерных игр [14].

Linux Ubuntu использует стабильную версию ядра *Linux* и имеет широкую поддержку всех необходимых драйверов и библиотек, что делает его отличным выбором для разработки и выполнения вычислительных экспериментов. Благодаря поддержке многоядерных процессоров и отличной производительности многозадачности, *Linux Ubuntu* становится хорошей платформой для интенсивных вычислений и разработки приложений, требующих высокой вычислительной мощности. Эта операционная система оптимизирована для быстрого старта и работы на устройствах с разными характеристиками, что позволяет эффективно использовать её даже на старых и маломощных машинах.

3 ТЕОРЕТИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ ПРОГРАММНОГО ПРОДУКТА

3.1 Обоснование необходимости разработки

В современных вычислительных системах производительность процессоров играет ключевую роль в обеспечении эффективного решения широкого круга задач — от прикладных вычислений до обработки больших объёмов данных. Многоядерные мобильные процессоры, такие как *Intel Core i7-12650H* и *AMD Ryzen 7 8845HS*, становятся стандартом в ноутбуках и мобильных рабочих станциях. Правильный выбор процессора, обеспечивающего оптимальное соотношение производительности и энергопотребления для конкретных приложений при ограниченном бюджете, представляет собой нетривиальную задачу. Для корректной оценки вычислительных возможностей процессоров требуется проведение контролируемых тестов и экспериментальных исследований.

Анализ производительности многоядерных процессоров на примере выполнения алгоритма пузырьковой сортировки является актуальной задачей по ряду причин:

1 Оптимизация вычислений — эффективное использование многоядерных платформ и настройка профилей энергопотребления может существенно сократить время выполнения алгоритмических задач; даже простые, но частые операции сортировки требуют внимания к организации памяти и кэшу, что особенно важно при работе с большими массивами данных.

2 Практическая применимость и диагностическая ценность — хотя пузырьковая сортировка является учебным и неэффективным по асимптотике алгоритмом, она широко используется в качестве эталонного теста для изучения влияния архитектуры процессора, кэш-политик и одноядерной производительности; результаты таких тестов помогают понять поведение аппаратной платформы в более сложных реальных алгоритмах и учебных задачах по обработке данных.

3 Конкурентоспособность платформ и адаптируемость ПО — в условиях быстро меняющегося рынка вычислительной техники показатели практической производительности становятся важным фактором при выборе оборудования для разработчиков, исследователей и конечных пользователей. Разработка и проверка оптимизированных реализаций сортировок и тестовых наборов для разных процессоров повышает их привлекательность и даёт основания для обоснованного выбора аппаратной платформы.

Важно отметить, что сравнительный анализ *Intel Core i7-12650H* и *AMD Ryzen 7 8845HS* на базе пузырьковой сортировки позволяет выявить особенности влияния частоты ядра, организации кэша, политики турбо-режимов и поведения планировщика ОС на практическое время выполнения.

Производительность этих процессоров напрямую влияет на скорость выполнения сортировок и эффективность использования системных ресурсов; понимание их сильных и слабых сторон важно для выбора аппаратной конфигурации под конкретные вычислительные задачи.

Таким образом, проведение контролируемых экспериментов и аналитической обработки результатов не только способствует технологическому прогрессу, но и предоставляет практические рекомендации по выбору и настройке аппаратной платформы. В рамках данной работы планируется выполнить следующие шаги: изучение технических характеристик процессоров, реализация алгоритма пузырьковой сортировки на C++, проведение вычислительных экспериментов, анализ полученных данных и построение графиков для наглядного сравнения производительности.

3.2 Технологии программирования, используемые для решения поставленных задач

Для реализации программного продукта, предназначенного для исследования производительности процессоров *Intel Core i7-12650H* и *AMD Ryzen 7 8845HS* при выполнении алгоритма пузырьковой сортировки, был выбран язык программирования C++. Этот выбор обусловлен высокой производительностью языка и возможностями тонкого управления памятью и низкоуровневой оптимизации, что критично при точном измерении времени выполнения алгоритмов и сравнении аппаратных платформ.

C++ обеспечивает минимальные накладные расходы и позволяет реализовать как классическую последовательную версию пузырьковой сортировки, так и её возможные варианты с распараллеливанием (например, параллельные проходы или вариант «*odd-even transposition*»), что даёт возможность исследовать влияние многопоточности и архитектурных особенностей процессоров на практическую скорость сортировки. Высокая производительность C++ делает его подходящим инструментом для реализации тестовых наборов с различными размерами входных массивов и точной настройкой алгоритмических параметров.

Для экспериментов с параллельными реализациями использовалась технология *OpenMP*, которая облегчает введение параллелизма на уровне директив и управления циклами (директивы `#pragma omp parallel`, `#pragma omp for` и т.д.). *OpenMP* позволяет гибко менять число потоков и структурировать параллельные регионы, что важно при сравнении поведения *i7-12650H* (гибридная архитектура P/E-ядер) и *Ryzen 7 8845HS* (традиционная многоядерная конфигурация). Использование *OpenMP* даёт возможность исследовать, как масштабируется время выполнения сортировки при увеличении числа потоков и как архитектурные различия влияют на эффективность распараллеливания.

Для точного измерения временных характеристик применялась

библиотека `<chrono>` из стандарта C++. Через `std::chrono::high_resolution_clock` и `std::chrono::duration_cast` выполнялось замеры времени выполнения сортировки для различных размеров массивов и конфигураций (последовательная/параллельная версии, разное число потоков). Точные измерения необходимы для корректного анализа и последующего построения графиков сравнения производительности.

Сохранение и последующая обработка результатов обеспечивались библиотекой *fstream*: результаты каждого прогона записывались в CSV-файлы (время выполнения, размер массива, число потоков, параметры привязки), что упрощает последующий статистический анализ и визуализацию данных.

Для контроля условий эксперимента и уменьшения влияния планировщика ОС использовались возможности Linux по управлению аффинити потоков. Привязка потоков к конкретным ядрам (*pthread_setaffinity_np*, использование *cpu_set_t* и макросов *CPU_SET*) применялась для фиксирования выполнения сортировок на Р- или Е-ядрах (в случае *i7-12650H*) либо на определённых физических ядрах (в случае *Ryzen*). Функция *setAffinityToCore* обеспечивает надёжную фиксацию выполнения потока на выбранном ядре или наборе ядер.

В результате, сочетание C++, *OpenMP*, *<chrono>*, *fstream* и механизмов управления аффинити в Linux обеспечивает все необходимые инструменты для реализации задач исследования: изучение технических характеристик процессоров, реализация последовательной и параллельной версий пузырьковой сортировки, проведение вычислительных экспериментов с контролируемыми параметрами, сбор и анализ данных, а также построение графиков для наглядного сравнения производительности рассматриваемых процессоров [15].

3.3 Связь архитектуры вычислительной системы с разрабатываемым программным обеспечением

Разрабатываемый программный продукт напрямую зависит от архитектурных особенностей исследуемых вычислительных платформ — *Intel Core i7-12650H* и *AMD Ryzen 7 8845HS*. Программа предусматривает запуск в нескольких режимах: классическая последовательная реализация пузырьковой сортировки и варианты с включённой/выключенной поддержкой гиперпоточности (*SMT/Hyper-Threading*), а также параллельные реализации (например, реализация по схеме *odd–even transposition*).

На уровне программной реализации взаимодействие с процессором осуществляется средствами операционной системы и библиотек параллелизма (*OpenMP*), а также посредством управления аффинити потоков в Linux. Это даёт возможность контролируемо распределять вычисления по физическим и логическим ядрам, фиксировать выполнение потоков на Р- или Е-ядрах (в случае *i7-12650H*) или на конкретных физических ядрах (в случае *Ryzen*), что

повышает воспроизводимость измерений и позволяет изучать влияние локализации данных в кэше на время выполнения сортировок.

Каждый из рассматриваемых процессоров имеет свои особенности многоядерной и многопоточной обработки. *Intel Core i7-12650H* использует гибридную организацию (*Performance* и *Efficient* ядра). *AMD Ryzen 7 8845HS* применяет традиционную многопоточную архитектуру с поддержкой SMT, что обеспечивает равномерное распределение потоков по физическим ядрам и высокую одноядерную частоту. Эти отличия определяют, как эффективно распараллеливаются отдельные этапы сортировки и насколько выигрыш даёт использование логических потоков вместо физических.

Для корректного понимания взаимодействия ПО с аппаратной платформой следует напомнить базовые понятия: физическое ядро — аппаратная единица, выполняющая инструкции; логическое ядро — представление для ОС, возникающее при использовании *SMT/Hyper-Threading*; процесс — изолированная сущность с собственным адресным пространством; поток — единица планирования, которой ОС выделяет процессорное время. Потоки программы создаются и управляются как средствами стандартной библиотеки потоков, так и средствами *OpenMP*.

Управление *CPU affinity* применяется для уменьшения «прыжков» исполнения между ядрами и лучшего использования кэша. В Linux это достигается с помощью системных вызовов (*pthread_setaffinity_np*, структура *cpu_set_t* и макросы *CPU_SET*). Фиксация потоков особенно важна при сравнительных экспериментах: она снижает влияние контекстных переключений и позволяет оценить, как поведение Р- и Е-ядер (у *i7*) или расположение потоков по физическим ядрам (у *Ryzen*) сказывается на времени выполнения пузырьковой сортировки.

Для параллельной версии сортировки используется *OpenMP*: директивы *#pragma omp parallel* и *#pragma omp for* облегчают распределение итераций циклов между потоками и позволяют гибко менять число активных потоков. Важно отметить, что классическая пузырьковая сортировка плохо распараллеливается в своей базовой форме; поэтому в работе рассматриваются как последовательная реализация, так и параллельные модификации (*odd-even transposition*, параллельные проходы по отдельным сегментам с последующей синхронизацией), которые лучше демонстрируют влияние архитектуры на масштабируемость алгоритма [16].

Таким образом, программное обеспечение взаимодействует с системой многопоточности и аффинити через ОС и *OpenMP*, что позволяет исследовать: влияние *SMT/Hyper-Threading* и гибридной архитектуры на производительность, поведение кэш-политик при работе с большими массивами, а также эффективность различных стратегий распараллеливания пузырьковой сортировки. Используемые подходы (последовательные и параллельные реализации, управление аффинити, точные замеры времени) обеспечат наглядную связь между архитектурой вычислительной системы и практическими результатами экспериментов. [17].

4 ПРОЕКТИРОВАНИЕ ФУНКЦИОНАЛЬНЫХ ВОЗМОЖНОСТЕЙ ПРОГРАММЫ

4.1 Общая цель программы

Цель программы заключается в проведении сравнительного анализа производительности двух процессоров — *Intel Core i7-12650H* и *AMD Ryzen 7 8845HS* — при выполнении алгоритма пузырьковой сортировки. Программа позволяет оценить, насколько эффективно рассматриваемые процессоры обрабатывают операции сортировки массивов различного размера и характера (случайные, отсортированные, инвертированные), а также выявить особенности их работы при разных условиях выполнения: последовательная реализация, оптимизированная версия с ранним выходом и параллельные модификации (например, *odd–even transposition*). В рамках исследования предусматривается анализ влияния числа активных ядер, режима гиперпоточности и привязки потоков на итоговое время сортировки.

4.2 Теоретические сведения о методе Гаусса

Алгоритмы сортировки относят к нескольким классам по различным признакам: по устойчивости (*stable/unstable*), по месту хранения (*in-place / out-of-place*), по типу подхода (*comparison-based / non-comparison*) и по асимптотической сложности. Пузырьковая сортировка — это простой *comparison-based* и *in-place* алгоритм, который относится к классу устойчивых сортировок в стандартной реализации и имеет квадратичную временную сложность в худшем и среднем случаях [18].

Пусть массив чисел задаётся вектором:

$$A = [a_1, a_2, \dots, a_n]. \quad (4.1)$$

Классическая реализация пузырьковой сортировки выполняет последовательные проходы по массиву, на каждом проходе сравнивая соседние элементы и, при необходимости, обменивая их местами, «всплывая» наибольшие элементы к концу массива. После k -го прохода гарантируется, что правые k элементов массива находятся в окончательно отсортированном порядке. Общая схема алгоритма можно представить так:

- 1 Для i от 1 до $n-1$ выполнить.
- 2 Для j от 1 до $n-i$ выполнить.
- 3 Если $A[j] > A[j+1]$, то обменять $A[j]$ и $A[j+1]$.

Асимптотика алгоритма: время выполнения в худшем и среднем случаях $O(n^2)$, в лучшем случае (если массив уже отсортирован и реализован условный ранний выход) $O(n)$. Алгоритм требует $O(1)$ дополнительной памяти (*in-place*).

Существуют варианты и оптимизации пузырьковой сортировки, которые важны для исследовательской части работы:

- ранний выход (если за проход не совершено ни одного обмена, алгоритм завершается досрочно);
- запоминание последней позиции обмена для уменьшения длины следующего прохода;
- параллельные модификации, в частности алгоритм *odd–even transposition*, который допускает распараллеливание операций сравнения/обмена по нечётным и чётным парам, что делает его удобным для оценки влияния многопоточности и архитектурных особенностей процессоров.

Для целей данной работы представляют интерес следующие свойства пузырьковой сортировки: высокая зависимость времени выполнения от одноядерной производительности и организации кэша при последовательной реализации; чувствительность к стратегиям распараллеливания и поведению планировщика ОС при параллельных вариантах; а также простота реализации, позволяющая фокусироваться на аппаратных отличиях (частоты, кэш, SMT/гибридные ядра, политика турбо-режима) при контролируемых экспериментах.

4.3 Описание функциональных возможностей

Программа предназначена для анализа производительности выполнения алгоритма пузырьковой сортировки на двух процессорах — *Intel Core i7-12650H* и *AMD Ryzen 7 8845HS*. Её функциональные возможности включают:

1 Генерация входных массивов различного типа — программа формирует массивы заданного размера: случайные, частично отсортированные, уже отсортированные и полностью инвертированные. Это позволяет моделировать разные практические сценарии сортировки и оценивать поведение алгоритма при различных входных данных.

2 Последовательная реализация пузырьковой сортировки — функция *bubbleSort* реализует классический *in-place* алгоритм пузырьковой сортировки без распараллеливания и с опцией раннего выхода (если за проход не было обменов), что служит базовой точкой отсчёта для сравнения с оптимизированными и параллельными версиями.

3 Оптимизированные варианты алгоритма — реализованы улучшения классической версии: ранний выход при отсутствии обменов, запоминание последней позиции обмена для сокращения длины следующего прохода, что уменьшает время выполнения на упорядоченных и почти упорядоченных массивах.

4 Параллельные модификации с *OpenMP* — реализованы параллельные варианты сортировки, в том числе подходы на основе параллельных проходов и алгоритма *odd–even transposition*, использующие директивы *#pragma omp parallel* и *#pragma omp for*. Это даёт возможность исследовать влияние

распараллеливания на *i7-12650H* (с учётом P/E-ядер) и *Ryzen 7 8845HS* (традиционная многопоточная архитектура).

5 Управление аффинити потоков — функция *setAffinityToCore* (использующая *pthread_setaffinity_np*, *cpu_set_t* и *CPU_SET*) позволяет закреплять потоки на конкретных ядрах или группах ядер. Это важно для получения воспроизводимых результатов и изучения влияния расположения потоков на P-/E-ядрах Intel и на физических ядрах AMD.

6 Многократное выполнение для минимизации системных отклонений — каждый тестовый прогон выполняется несколько раз (например, 10 прогонов), после чего берётся среднее или медианное значение времени. Такой подход снижает влияние фоновой загрузки системы и случайных флуктуаций на результаты.

7 Поддержка трёх режимов запуска — последовательный режим, параллельный режим (*OpenMP*) и режим с привязкой потоков к ядрам. Режим параллельного выполнения позволяет измерить масштабируемость и ускорение при увеличении числа потоков; режим с привязкой к ядру даёт «чистые» однопоточные замеры без влияния миграции потоков.

8 Замеры времени и управление профилями — для точных измерений используется `<chrono>` (`std::chrono::high_resolution_clock`). Также предусмотрена возможность изменения профиля энергопотребления / TDP (через настройки BIOS/системы) и конфигурации числа активных потоков для изучения их влияния на время сортировки.

9 Сохранение результатов и расчёт ускорения — все результаты (время выполнения, размер массива, тип данных, число потоков, режим аффинити, параметры запуска) записываются в CSV-файлы. Программа вычисляет относительное ускорение (*speedup*) параллельных и оптимизированных версий относительно базовой последовательной реализации и сохраняет эти данные для последующего анализа.

10 Автоматическое тестирование по диапазону размеров — программа автоматически прогоняет тесты для множества размеров массивов (малые, средние, большие), что позволяет построить кривые зависимости времени от размера входных данных и оценить поведение алгоритма при росте объёма задач.

В целом, реализованный набор функциональных возможностей обеспечивает полный цикл исследования: изучение характеристик платформ (с помощью настроек аффинити и параметров запуска), разработка и тестирование последовательных и параллельных реализаций пузырьковой сортировки на C++, проведение воспроизводимых экспериментов, сбор и сохранение данных для дальнейшего анализа, а также построение графиков и оценку масштабируемости и ускорения для сравнительного анализа *Intel Core i7-12650H* и *AMD Ryzen 7 8845HS*.

5 СРАВНЕНИЕ ПРОИЗВОДИТЕЛЬНОСТИ ПРОЦЕССОРОВ

5.1 Общая структура программы

Программа реализует алгоритм пузырьковой сортировки на языке C++. Она предоставляет как последовательную, так и параллельную версии алгоритма с использованием *OpenMP*, что позволяет задействовать все доступные логические ядра процессора и исследовать масштабируемость на многоядерных платформах. Для упрощения анализа результатов программа выводит результаты в консоль, сохраняет их в файл и поддерживает режим запуска на одном ядре для проверки однопоточной производительности. Листинг исходного кода приведён в приложении Б.

Основная структура данных в программе — одномерный массив, который представлен в виде `std::vector<int>` (или `std::vector<double>` при необходимости). Для создания входных данных используется генератор случайных чисел с равномерным распределением; дополнительно поддерживается генерация частично упорядоченных и инвертированных массивов. Это позволяет тестировать сортировку на массивах разных типов и размеров, начиная от небольших и заканчивая крупными, чтобы выявить зависимость времени выполнения от объёма и характера входных данных.

Для компиляции и выполнения используются стандартные библиотеки `<iostream>`, `<vector>`, `<chrono>`, `<iomanip>`, `<fstream>`; для управления привязкой потоков к ядрам в среде Linux применяется *POSIX Threads* (`<pthread.h>`). Поддержка *OpenMP* (`<omp.h>`) обеспечивает удобное и гибкое распараллеливание алгоритма. Комбинация этих инструментов позволяет как запускать чисто последовательные измерения, так и исследовать поведение параллельных реализаций на платформах *Intel Core i7-12650H* и *AMD Ryzen 7 8845HS*.

Классическая реализация пузырьковой сортировки последовательно проходит по массиву, сравнивая соседние элементы и обменивая их при необходимости, что требует $O(n^2)$ времени в среднем и худшем случае. Параллельная реализация может быть выполнена несколькими способами (например, распараллеливание независимых сегментов, использование алгоритма *odd–even transposition*), и в программе предусмотрены варианты параллельной реализации, позволяющие исследовать влияние числа потоков и стратегии распределения итераций на итоговое время выполнения.

В параллельной части используются директивы *OpenMP* `#pragma omp parallel` и `#pragma omp for` для распределения итераций по потокам; при этом при необходимости применяется `schedule(static)` для статического распределения участков работы между потоками — выбор расписания определяется экспериментально для минимизации накладных расходов планирования и обеспечения равномерной загрузки. Для некоторых

параллельных модификаций (например, *odd–even*) синхронизация между шагами реализуется с помощью барьеров *OpenMP*, что обеспечивает корректность и воспроизводимость результатов.

Для тестирования в однопоточном режиме используется привязка потока к одному ядру с помощью *POSIX Threads*. Функция *pthread_setaffinity_np* закрепляет выполнение процесса или потока за указанным ядром процессора; это позволяет исключить влияние миграции потоков между ядрами (что может приводить к потерям в кэше и дополнительным контекстным переключениям) и получить «чистые» замеры однопоточной производительности.

Структура программы включает шаблонные и утилитные функции для измерения времени выполнения каждой реализации: замеры осуществляются с помощью *std::chrono::high_resolution_clock*, результаты усредняются по нескольким запускам (например, 10 прогонов) для снижения влияния фоновой нагрузки и случайных флуктуаций. Также реализованы механизмы логирования и сохранения данных — каждое измерение (время выполнения, размер массива, тип входных данных, число потоков, режим аффинити и прочие параметры) записывается в CSV-файл, что упрощает последующий статистический анализ и построение графиков.

Общая архитектура программы обеспечивает полный цикл исследований: генерация тестовых массивов, реализация последовательной и параллельной версий пузырьковой сортировки на C++, проведение воспроизводимых экспериментов с контролем аффинити и числа потоков, сбор и сохранение результатов для последующего анализа и визуализации.

5.2 Описание функциональной схемы программы

Функциональная схема программы представлена в приложении В. В ходе разработки схемы выделены следующие ключевые блоки:

1 Инициализация параметров тестирования. На этом этапе выполняется настройка основных параметров программы: количество итераций для прогонов, набор размеров тестируемых массивов, типы генерируемых входных данных (случайные, частично отсортированные, уже отсортированные, инвертированные), идентификатор ядра для однопоточного теста и параметры распараллеливания (максимальное число потоков). Инициализируются структуры для хранения результатов и логирования.

2 Тестирование функций. Тестирование заключается в измерении времени выполнения трёх вариантов реализации алгоритма пузырьковой сортировки: последовательной реализации (*bubbleSort*), оптимизированной последовательной реализации (ранний выход, запоминание последней позиции обмена) и параллельной реализации с использованием *OpenMP* (*bubbleSort_OMP*, включая вариант *odd–even transposition*). В однопоточном режиме тесты выполняются с привязкой потока к заданному ядру (*setAffinityToCore*), что позволяет получить воспроизводимые замеры без

миграции по ядрам. Для каждого варианта алгоритма и для каждого размера/типа массива выполняется несколько итераций; результаты (время каждого прогона) накапливаются в структуре для дальнейшего анализа. [19].

3 Вывод результатов в консоль. На этом этапе производится форматированный вывод агрегированных результатов в консоль в виде таблицы с колонками: размер массива (*Size*), тип массива (*Type*), время последовательного метода (*Sequential*), время оптимизированного метода (*Optimized*), время параллельного метода (*OpenMP*), время выполнения при привязке к одному ядру (*Single Core*), а также расчётные показатели ускорения (*speedup*) для параллельных и оптимизированных версий относительно базовой последовательной реализации.

4 Запись результатов в файл. Все результаты тестирования сохраняются в CSV-файлы (время выполнения, размер массива, тип входных данных, число потоков, режим аффинити, параметры запуска). Это необходимо для дальнейшей обработки, статистического анализа и построения графиков, демонстрирующих зависимость времени выполнения от размера массива, типа входных данных и числа потоков.

5 Многократные прогоны и агрегация. Для уменьшения влияния фоновой загрузки и случайных флуктуаций каждый набор параметров прогоняется множеством запусков (например, 10 прогонов). Программа вычисляет средние, медианные и стандартные отклонения времени, что позволяет строить более надёжные графики и делать корректные выводы о сравнительной производительности *Intel Core i7-12650H* и *AMD Ryzen 7 8845HS*.

6 Управление профилями и параметрами платформы. Программа предоставляет интерфейс для изменения числа активных потоков, включения/отключения *SMT/Hyper-Threading* (возможности ОС/BIOS), а также для задания параметров привязки потоков, что требуется при экспериментальном сравнении архитектур *i7-12650H* (P-/E-ядра) и *Ryzen 7 8845HS*.

5.3 Описание блок-схемы алгоритма программы

Блок-схема алгоритма программы представлена в приложении Г. Схема включает следующие ключевые функции и блоки:

- *main*: начальная точка программы; парсит параметры запуска, инициализирует списки размеров и типов массивов, последовательно вызывает тестирующие процедуры, собирает и сохраняет результаты, формирует итоговые CSV-файлы и сводные таблицы для вывода и визуализации;

- *test*: универсальная функция измерения производительности; принимает в качестве аргумента указатель/функтор на конкретную реализацию сортировки, запускает её заданное число итераций на одном и том же входном массиве (или на наборе массивов), усредняет результаты и

возвращает агрегированные метрики (среднее, медиана, стандартное отклонение);

- *generateArray*: генерирует входной массив заданного размера и типа (*random*, *nearly-sorted*, *sorted*, *reversed*). Для воспроизводимости тестов предусмотрена возможность задания *seed* для генератора случайных чисел;

- *bubbleSort*: реализует классическую последовательную версию пузырьковой сортировки (*in-place*) с опцией раннего выхода;

- *bubbleSortOptimized*: реализует версии с оптимизациями (запоминание последнего обмена, уменьшение области прохода), используемые для оценки влияния простых оптимизаций на одноядерную производительность.

5.4 Подготовка к тестированию

Для анализа производительности выполнения алгоритма пузырьковой сортировки на платформах *Intel Core i7-12650H* и *AMD Ryzen 7 8845HS* будет проведено комплексное тестирование в различных режимах загрузки. Основной акцент сделан на сравнении однопоточной производительности и поведения при полной загрузке всех доступных логических ядер. Испытания планируется проводить в два этапа: с включённой гиперпоточностью (*Hyper-Threading/SMT*) и с отключённой, чтобы оценить влияние этого фактора на эффективность сортировки и масштабируемость параллельных модификаций алгоритма. Для получения детальной картины поведения архитектур также собираются аппаратные метрики производительности.

Тестирование будет выполняться в трёх режимах:

- последовательная реализация алгоритма без использования *OpenMP*;
- параллельная реализация с использованием *OpenMP* (включая варианты *odd-even transposition* и распараллеливание проходов);

- последовательный запуск с привязкой потока к одному ядру (*single-core*) для «чистых» однопоточных измерений.

Входные данные: для тестов генерируются массивы нескольких типов (случайные, частично упорядоченные, полностью упорядоченные, инвертированные) и различных размеров. Диапазон размеров массивов подобран таким образом, чтобы охватить малые, средние и большие задачи, демонстрирующие квадратичную сложность алгоритма и влияние кэша:

- от 10 до 500 элементов с шагом 10;
- от 1 000 до 100 000 элементов с шагом 1 000;
- от 200 000 до 1 000 000 элементов с шагом 100 000.

Каждый прогон выполняется многократно (например, 10 раз) и для анализа используется среднее (или медианное) значение времени, что снижает влияние фоновой загрузки и случайных системных флуктуаций.

Для сбора аппаратных метрик на уровне процессора используется утилита *perf* [20]. Для каждого прогона и режима измеряются следующие показатели:

- общее количество тактов процессора (*cycles*);

- количество исполненных инструкций (*instructions*);
- обращения к кэш-памяти (*cache-references*);
- промахи кэша (*cache-misses*);
- количество переходов в коде (*branches*);
- промахи предсказания переходов (*branch-misses*);
- количество *page-faults*;
- контекстные переключения (*context-switches*);
- застой на этапе выборки инструкций (*stalled-cycles-frontend*).

Сбор метрик и логирование выполняются отдельными запусками тестов для каждого режима и каждой конфигурации (число потоков, привязка к ядрам, тип массива, размер). Это позволяет вывести корреляции между аппаратными событиями (например, числом промахов кэша или *branch-misses*) и наблюдаемым временем выполнения сортировки.

Последовательность экспериментов следующая: сначала проводятся тесты с включённой гиперпоточностью на обоих устройствах, затем *Hyper-Threading/SMT* отключается на уровне BIOS/платформы, и тесты повторяются для получения сопоставимого набора данных. Все результаты (время выполнения, аппаратные счётчики, параметры запуска) сохраняются в CSV для дальнейшего статистического анализа и построения графиков зависимости времени от размера массива, ускорения при распараллеливании и влияния архитектурных особенностей на производительность.

5.5 Результаты измерений с включенным Hyper-Threading/SMT

Таблица 5.1 представляет замеры времени для программы, использующей пузырьковую сортировку на процессоре *AMD Ryzen 7 8845HS* с включенной функцией *SMT*. Колонка размер содержит размерность матрицы, для которой были проведены замеры. В колонке последовательный указано время выполнения (в миллисекундах) для последовательной реализации пузырьковой сортировки, а в колонке параллельный (*OpenMP*) – для параллельной версии решения той же задачи только с использованием *OpenMP*, распределяющего нагрузку по всем доступным ядрам. Колонка на одном ядре показывает время выполнения при запуске, закрепленном за одним ядром.

Таблица 5.1 – Замеры времени на *AMD Ryzen 7 8845HS*

Размер	Последовательный	Параллельный (<i>OpenMP</i>)	На одном ядре
1000	1,65	10,23	1,55
2000	5,88	3,08	6,07
3000	12,88	4,75	13,19
4000	22,80	6,50	23,38
5000	35,71	8,21	36,29

Продолжение таблицы 5.1

18000	451,07	34,32	454,91
21000	606,90	41,91	616,43
24000	796,12	50,92	797,12
27000	1022,78	60,49	1015,76
30000	1263,68	70,79	1239,14
33000	1539,69	82,31	1504,63
36000	1826,36	93,62	1788,39
39000	2120,76	107,15	2093,01
42000	2483,52	120,39	2428,15
45000	2804,78	134,63	2786,01
48000	3210,21	151,59	3169,33
72000	7109,59	319,10	7151,65
75000	7695,18	343,44	7784,77
78000	8345,22	365,02	8440,33
81000	9019,88	386,72	9127,45
84000	9734,02	410,86	9851,49
87000	10483,15	439,91	10608,15
90000	11275,40	470,63	11415,82
93000	12080,90	502,31	12220,20
96000	12995,80	533,22	13143,98
99000	14002,83	553,95	14159,22

Таблица 5.2 представляет замеры времени для программы, реализующей пузырьковую сортировку на процессоре *Intel Core i7-12650H* с включенной функцией *Hyper-Threading*.

Таблица 5.2 – Замеры времени на *Intel Core i7-12650H*

Размер	Последовательный	Параллельный (<i>OpenMP</i>)	На одном ядре
1000	1,41	14,18	1,58
2000	5,29	18,44	6,08
3000	11,05	39,10	11,49
4000	19,31	10,13	19,48
5000	29,92	13,56	30,22
18000	497,98	124,57	508,82
21000	701,36	119,68	715,93
24000	934,09	209,44	952,90
27000	1202,59	411,85	1226,88

Продолжение таблицы 5.2

30000	1502,46	271,80	1531,08
33000	1866,78	1300,06	1870,06
36000	2317,97	545,99	2241,33
39000	2687,97	318,29	2649,11
42000	3072,60	792,63	3094,40
45000	3622,98	691,75	3576,16
48000	4057,82	990,12	4090,97
72000	9377,43	2946,61	9410,16
75000	10241,09	10643,40	10242,43
78000	11016,40	4106,30	11113,00
81000	12021,40	6203,46	11991,40
84000	12787,96	3146,57	12923,78
87000	13765,01	4773,90	13882,83
90000	14564,02	2688,91	14887,71
93000	15540,68	4088,53	15878,38
96000	16589,65	3854,49	16969,12
99000	17676,05	6622,15	18100,10

Также были проведены измерения ускорения решения с помощью *OpenMP* и привязки к конкретному ядру по сравнению с обычной пузырьковой сортировкой, которые показывают, насколько быстрее или медленнее сработала конкретная вариация решения. Рассчитывается как отношение времени выполнения последовательной версии алгоритма к времени выполнения параллельной версии или версии с привязкой к одному ядру. Результаты этих измерений для *AMD Ryzen 7 8845HS* приведены в таблице 5.3.

Таблица 5.3 – Замеры ускорения на *AMD Ryzen 7 8845HS*

Размер	Ускорение <i>OpenMP</i>	Ускорение <i>Single Core</i>
1000	0,161255623	1,065934066
2000	1,907911803	0,968719131
3000	2,709718132	0,976574937
4000	3,510238645	0,97506629
5000	4,350919946	0,983935411
18000	13,14141708	0,991547797
21000	14,47972038	0,984551268
24000	15,63525669	0,998745477
27000	16,90718088	1,006917009
30000	17,85236985	1,019803235

Продолжение таблицы 5.3

33000	18,70535638	1,023306108
36000	19,50778663	1,021227984
39000	19,79166433	1,01325457
42000	20,62964132	1,022804211
45000	20,83278988	1,006738677
48000	21,1771974	1,012898949
72000	22,28049014	0,994119954
75000	22,40617575	0,988491754
78000	22,86241264	0,988731477
81000	23,32387781	0,988214996
84000	23,6917668	0,988076421
87000	23,83036869	0,988216323
90000	23,95789509	0,987699958
93000	24,05073371	0,988600841
96000	24,37211378	0,98872647
99000	25,27796893	0,988954902

Аналогичные измерения были проведены и на процессоре *Intel Core i7-12650H*. Результаты приведены в таблице 5.4

Таблица 5.4 – Замеры ускорения на *Intel Core i7-12650H*

Размер	Ускорение <i>OpenMP</i>	Ускорение <i>Single Core</i>
1000	0,099753086	0,893240682
2000	0,286853238	0,869758263
3000	0,28246637	0,961438022
4000	1,905824284	0,990864299
5000	2,206784661	0,99023793
18000	3,997728112	0,978703542
21000	5,860285762	0,97964882
24000	4,459867362	0,980260217
27000	2,919956489	0,980201764
30000	5,527842266	0,981311182
33000	1,435922326	0,998246578
36000	4,24543673	1,034194505
39000	8,445070361	1,014669827
42000	3,876464429	0,992955983
45000	5,237374847	1,013089724
48000	4,098298997	0,99189484

Продолжение таблицы 5.4

72000	3,182450126	0,996522481
75000	0,962201541	0,999869953
78000	2,682802588	0,991307926
81000	1,93785323	1,00250171
84000	4,064092268	0,989490846
87000	2,883391235	0,991513184
90000	5,416333694	0,97825791
93000	3,801046832	0,978731957
96000	4,303986395	0,977637846
99000	2,669232199	0,976571896

Для анализа производительности различных реализаций пузырьковой сортировки были проведены измерения аппаратных метрик с использованием утилиты *perf*. Эти метрики отражают некоторые аспекты работы процессора, такие как количество тактовых циклов, выполненных инструкций, промахов кэша, переключений контекста и так далее. В таблице 5.5 представлены результаты измерений для каждого из исследуемых вариантов решения на процессоре *AMD Ryzen 7 8845HS*.

Таблица 5.5 – Замеры метрик на *AMD Ryzen 7 8845HS*

<i>N</i>	Последовательный	Параллельный (<i>OpenMP</i>)	На одном ядре
<i>cycles</i>	71 200 267 575	37 757 075 879	70 874 776 450
<i>instructions</i>	51 624 390 441	50 427 296 102	51 587 687 903
<i>insn per cycle</i>	0,73	1,34	0,73
<i>cache references</i>	327 180 167	750 519 037	349 235 308
<i>cache-misses</i>	1 655 362	31 354 912	2 626 733
<i>branches</i>	9 843 327 195	10 101 170 324	9 829 636 958
<i>branch misses</i>	120 407 004	20 903 106	113 146 149
<i>seconds time elapsed</i>	14,26405169	0,514620464	14,34177515
<i>seconds user</i>	14,147926	8,097699	14,231334
<i>seconds sys</i>	0,002969	0,022833	0,001962

Аналогичные измерения метрик при выполнении задач были проведены и на процессоре *Intel Core i7-12650H*. Все метрики, доступные на процессоре *AMD Ryzen 7 8845HS*, поддерживаются на процессоре *Intel*. В таблице 5.6 представлены результаты измерений для каждого из исследуемых вариантов решения на процессоре *Intel Core i7-12650H*.

Таблица 5.6 – Замеры метрик на *Intel Core i7-12650H*

<i>N</i>	Последовательный	Параллельный (<i>OpenMP</i>)	На одном ядре
<i>cycles</i>	82 556 565 115	331 870 109 245	83 339 867 220
<i>instructions</i>	51 522 562 568	61 676 586 332	51 501 138 324
<i>insn per cycle</i>	0,62	0,19	0,62
<i>cache references</i>	1 347 955	50 800 508	3 816 201
<i>cache-misses</i>	112 210	1 583 367	287 894
<i>branches</i>	9 811 711 067	13 333 789 969	9 810 006 158
<i>branch misses</i>	1 217 751 504	249 679 408	1 248 859 789
<i>seconds time elapsed</i>	17,87343033	6,0555779	18,45833121
<i>seconds user</i>	17,868863	87,408502	18,451579
<i>seconds sys</i>	0,003999	0,076038	0,003999

5.6 Результаты измерений с выключенным Hyper-Threading/SMT

Теперь рассмотрим результаты измерений производительности с выключенной гиперпоточностью (*Hyper-Threading/SMT*). Ниже будут представлены аналогичные замеры, как и при включенной функции гиперпоточности.

Таблица 5.7 представляет замеры времени для программы, реализующей пузырьковую сортировку на процессоре *AMD Ryzen 7 8845HS* с выключенной функцией *SMT*.

Таблица 5.7 – Замеры времени на *AMD Ryzen 7 8845HS*

Размер	Последовательный	Параллельный (<i>OpenMP</i>)	На одном ядре
1000	1,46	8,24	1,53
2000	5,76	2,90	6,04
3000	12,87	3,71	13,22
4000	22,81	5,16	23,36
5000	35,27	6,69	36,23
18000	449,61	30,61	454,17
21000	606,39	38,60	611,88
24000	793,72	46,69	802,03
27000	997,98	55,43	1008,08
30000	1234,03	65,68	1246,88
33000	1487,48	76,04	1502,35
36000	1769,13	87,26	1787,42

Продолжение таблицы 5.7

39000	2070,67	100,81	2089,89
42000	2400,94	112,71	2426,12
45000	2757,08	126,71	2784,41
48000	3134,20	141,59	3165,17
72000	7064,47	305,51	7284,12
75000	7697,43	328,94	7830,77
78000	8346,53	352,26	8444,09
81000	9024,13	374,64	9131,27
84000	9745,93	400,35	9853,77
87000	10492,18	427,23	10611,33
90000	11289,23	452,75	11420,74
93000	12098,29	480,55	12227,98
96000	13013,87	509,57	13152,88
99000	13999,92	534,52	14170,62

Таблица 5.8 представляет замеры времени для программы на процессоре *Intel Core i7-12650H* с выключенной функцией *Hyper-Threading*.

Таблица 5.8 – Замеры времени на *Intel Core i7-12650H*

Размер	Последовательный	Параллельный (<i>OpenMP</i>)	На одном ядре
1000	3,182	4,65	1,61
2000	5,318	6,665	6,15
3000	10,917	8,554	11,567
4000	19,434	11,093	19,449
5000	30,303	13,635	30,255
18000	521,906	76,621	509,866
21000	717,672	79,973	714,591
24000	988,336	102,92	952,608
27000	1249,549	118,102	1228,58
30000	1581,343	138,855	1536,111
33000	1860,956	197,345	1873,294
36000	2201,091	201,558	2251,238
39000	2639,492	189,855	2659,909
42000	3113,572	287,166	3104,74
45000	3517,014	353,317	3583,002
48000	4150,286	310,208	4099,699
72000	9387,948	819,726	9493,663
75000	10028,453	914,28	10263,768

Продолжение таблицы 5.8

78000	10874,131	1032,637	11126,138
81000	11719,621	1058,406	11998,486
84000	12656,177	1156,935	12949,399
87000	13857,85	1253,188	13895,052
90000	14552,512	1200,383	14909,209
93000	15564,01	1448,267	15895,953
96000	16581,926	1520,509	16988,831
99000	17679,527	1637,541	18087,707

Результаты измерения ускорения на *AMD Ryzen 7 8845HS* с выключенным *SMT* приведены в таблице 5.9.

Таблица 5.9 – Замеры ускорения на *AMD Ryzen 7 8845HS*

Размер	Ускорение <i>OpenMP</i>	Ускорение <i>Single Core</i>
1000	0,177427184	0,954308094
2000	1,985871813	0,954297069
3000	3,467403017	0,97374792
4000	4,418911064	0,976326041
5000	5,273773923	0,973557095
18000	14,68840248	0,989972829
21000	15,70884928	0,991035795
24000	17,00017135	0,989635103
27000	18,00328324	0,989975022
30000	18,78907701	0,989691094
33000	19,56174382	0,990100163
36000	20,27516732	0,989769589
39000	20,54070113	0,990801434
42000	21,30248343	0,989622916
45000	21,75865743	0,990185001
48000	22,13637648	0,990216317
72000	23,12321858	0,969844551
75000	23,40051012	0,982973062
78000	23,69401839	0,988446469
81000	24,08779746	0,988267453
84000	24,3434711	0,989056471
87000	24,55880363	0,988771906
90000	24,93479845	0,988485157
93000	25,17587103	0,98939432

Продолжение таблицы 5.9

96000	25,53872647	0,989431363
99000	26,19148136	0,987954161

Аналогичные измерения проведены на процессоре *Intel Core i7-12650H*.
Результаты приведены в таблице 5.10

Таблица 5.10 – Замеры ускорения на *Intel Core i7-12650H*

Размер	Ускорение <i>OpenMP</i>	Ускорение <i>Single Core</i>
1000	0,684301075	1,976397516
2000	0,797899475	0,864715447
3000	1,276245032	0,943805654
4000	1,751915622	0,999228752
5000	2,222442244	1,001586515
18000	6,811526866	1,023614048
21000	8,973928701	1,004311557
24000	9,60295375	1,037505459
27000	10,58025266	1,017067672
30000	11,38844838	1,029445789
33000	9,429962756	0,993413741
36000	10,9203852	0,977724701
39000	13,90267309	0,992324173
42000	10,84241171	1,002844683
45000	9,954273358	0,981583041
48000	13,37904245	1,012339199
72000	11,4525439	0,988864677
75000	10,96868902	0,977073235
78000	10,53044874	0,977350002
81000	11,07289736	0,976758318
84000	10,93940195	0,977356324
87000	11,05807748	0,997322644
90000	12,123224	0,976075391
93000	10,74664409	0,979117767
96000	10,90550993	0,976048676
99000	10,79638739	0,977433292

В таблице 5.11 представлены результаты измерений метрик для каждого из исследуемых вариантов реализации пузырьковой сортировки на процессоре *AMD Ryzen 7 8845HS*.

Таблица 5.11 – Замеры метрик на *AMD Ryzen 7 8845HS*

<i>N</i>	Последовательный	Параллельный (<i>OpenMP</i>)	На одном ядре
<i>cycles</i>	71 070 962 912	18 432 011 839	70 878 529 763
<i>instructions</i>	51 580 579 720	49 637 391 687	51 558 399 707
<i>insn per cycle</i>	0,73	2,69	0,73
<i>cache references</i>	321 809 742	681 668 331	320 353 758
<i>cache-misses</i>	447 633	15 366 364	325 248
<i>branches</i>	9 827 832 691	9 946 116 100	9 830 094 813
<i>branch misses</i>	118 439 706	13 100 348	115 452 012
<i>seconds time elapsed</i>	14,22024313	0,498945417	14,4243322
<i>seconds user</i>	14,13264	3,923831	14,331591
<i>seconds sys</i>	0,000989	0,017896	0,000999

Аналогичные измерения были проведены и на тестируемом процессоре *Intel Core i7-12650H*, результаты этих измерений продемонстрированы в таблице 5.12.

Таблица 5.12 – Замеры метрик на *Intel Core i7-12650H*

<i>N</i>	Последовательный	Параллельный (<i>OpenMP</i>)	На одном ядре
<i>cycles</i>	82 508 125 817	44 829 999 388	82 721 410 635
<i>instructions</i>	51 475 597 225	36 083 732 005	51 496 826 508
<i>insn per cycle</i>	0,62	0,8	0,62
<i>cache references</i>	411 508	15 301 685	427 729
<i>cache-misses</i>	68 839	96 085	75 512
<i>branches</i>	9 809 145 147	7 301 784 792	9 809 244 726
<i>branch misses</i>	1 226 402 148	86 488 403	1 222 169 988
<i>seconds time elapsed</i>	17,62761336	1,69112593	18,06169529
<i>seconds user</i>	17,627038	16,565411	18,053044
<i>seconds sys</i>	0	0,027955	0,003998

Для сравнения разницы между выполнением задачи на обоих процессорах (при включенной гиперпоточности) с использованием *OpenMP* была найдена разницы между замерами времени на *Intel Core i7-12650H* и *AMD Ryzen 7 8845HS* (результаты на *Intel* минус результаты на *AMD*).

Результаты приведены в таблице 5.13

Таблица 5.13 – Сравнение времени для метода с *OpenMP*

Размер	<i>Ryzen 7 8845HS</i>	<i>Intel Core I7 12650H</i>	Разница
1000	10,23	14,18	-3,95
2000	3,08	18,44	-15,35
3000	4,75	39,10	-34,35
4000	6,50	10,13	-3,64
5000	8,21	13,56	-5,35
18000	34,32	124,57	-90,24
21000	41,91	119,68	-77,77
24000	50,92	209,44	-158,53
27000	60,49	411,85	-351,36
30000	70,79	271,80	-201,01
33000	82,31	1300,06	-1217,74
36000	93,62	545,99	-452,37
39000	107,15	318,29	-211,14
42000	120,39	792,63	-672,24
45000	134,63	691,75	-557,12
48000	151,59	990,12	-838,53
72000	319,10	2946,61	-2627,51
75000	343,44	10643,40	-10299,96
78000	365,02	4106,30	-3741,29
81000	386,72	6203,46	-5816,74
84000	410,86	3146,57	-2735,71
87000	439,91	4773,90	-4333,99
90000	470,63	2688,91	-2218,27
93000	502,31	4088,53	-3586,22
96000	533,22	3854,49	-3321,26
99000	553,95	6622,15	-6068,19

Аналогичные результаты для последовательного метода при включенной гиперпоточности приведены в таблице в таблице 5.14. Здесь тоже была найдена разницы между замерами времени на *Intel Core i7-12650H* и *AMD Ryzen 7 8845HS*

Таблица 5.14 – Сравнение времени для последовательного метода

Размер	<i>Ryzen 7 8845HS</i>	<i>Intel Core I7 12650H</i>	Разница
1000	1,65	1,41	0,24
2000	5,88	5,29	0,60
3000	12,88	11,05	1,84
4000	22,80	19,31	3,49
5000	35,71	29,92	5,78
18000	451,07	497,98	-46,92
21000	606,90	701,36	-94,46
24000	796,12	934,09	-137,97
27000	1022,78	1202,59	-179,80
30000	1263,68	1502,46	-238,78
33000	1539,69	1866,78	-327,08
36000	1826,36	2317,97	-491,61
39000	2120,76	2687,97	-567,22
42000	2483,52	3072,60	-589,08
45000	2804,78	3622,98	-818,19
48000	3210,21	4057,82	-847,61
72000	7109,59	9377,43	-2267,84
75000	7695,18	10241,09	-2545,92
78000	8345,22	11016,40	-2671,19
81000	9019,88	12021,40	-3001,52
84000	9734,02	12787,96	-3053,94
87000	10483,15	13765,01	-3281,86
90000	11275,40	14564,02	-3288,62
93000	12080,90	15540,68	-3459,78
96000	12995,80	16589,65	-3593,86
99000	14002,83	17676,05	-3673,22

В таблице 5.15 приведены вычисления для метода с привязкой к одному ядру с включенными функциями *SMT/Hyper-Threading*.

Таблица 5.15 – Сравнение времени для метода с привязкой к одному ядру

Размер	<i>Ryzen 7 8845HS</i>	<i>Intel Core I7 12650H</i>	Разница
1000	1,55	1,58	-0,04

Продолжение таблицы 5.15

2000	6,07	6,08	-0,01
3000	13,19	11,49	1,70
4000	23,38	19,48	3,90
5000	36,29	30,22	6,07
18000	454,91	508,82	-53,91
21000	616,43	715,93	-99,50
24000	797,12	952,90	-155,78
27000	1015,76	1226,88	-211,12
30000	1239,14	1531,08	-291,94
33000	1504,63	1870,06	-365,43
36000	1788,39	2241,33	-452,93
39000	2093,01	2649,11	-556,10
42000	2428,15	3094,40	-666,25
45000	2786,01	3576,16	-790,16
48000	3169,33	4090,97	-921,65
72000	7151,65	9410,16	-2258,51
75000	7784,77	10242,43	-2457,66
78000	8440,33	11113,00	-2672,67
81000	9127,45	11991,40	-2863,95
84000	9851,49	12923,78	-3072,29
87000	10608,15	13882,83	-3274,68
90000	11415,82	14887,71	-3471,90
93000	12220,20	15878,38	-3658,18
96000	13143,98	16969,12	-3825,14
99000	14159,22	18100,10	-3940,88

Такие же расчеты с вычислением разницы были проведены для всех трех методов при выключенной функции гиперпоточности. Результаты этих измерений приведены в таблицах 5.16-5.18.

Таблица 5.16 – Сравнение времени для метода с *OpenMP*

Размер	<i>Ryzen 7 8845HS</i>	<i>Intel Core I7 12650H</i>	Разница
1000	8,24	4,65	3,59
2000	2,90	6,665	-3,76
3000	3,71	8,554	-4,84
4000	5,16	11,093	-5,93
5000	6,69	13,635	-6,95

Продолжение таблицы 5.16

18000	30,61	76,621	-46,01
21000	38,60	79,973	-41,37
24000	46,69	102,92	-56,23
27000	55,43	118,102	-62,67
30000	65,68	138,855	-73,18
33000	76,04	197,345	-121,31
36000	87,26	201,558	-114,30
39000	100,81	189,855	-89,05
42000	112,71	287,166	-174,46
45000	126,71	353,317	-226,61
48000	141,59	310,208	-168,62
72000	305,51	819,726	-514,21
75000	328,94	914,28	-585,34
78000	352,26	1032,637	-680,37
81000	374,64	1058,406	-683,77
84000	400,35	1156,935	-756,58
87000	427,23	1253,188	-825,96
90000	452,75	1200,383	-747,63
93000	480,55	1448,267	-967,72
96000	509,57	1520,509	-1010,94
99000	534,52	1637,541	-1103,02

Таблица 5.17 – Сравнение времени для последовательного метода

Размер	<i>Ryzen 7 8845HS</i>	<i>Intel Core I7 12650H</i>	Разница
1000	1,46	3,182	-1,72
2000	5,76	5,318	0,45
3000	12,87	10,917	1,95
4000	22,81	19,434	3,37
5000	35,27	30,303	4,97
18000	449,61	521,906	-72,29
21000	606,39	717,672	-111,28
24000	793,72	988,336	-194,62
27000	997,98	1249,549	-251,57
30000	1234,03	1581,343	-347,31
33000	1487,48	1860,956	-373,48
36000	1769,13	2201,091	-431,96

Продолжение таблицы 5.17

39000	2070,67	2639,492	-568,83
42000	2400,94	3113,572	-712,63
45000	2757,08	3517,014	-759,93
48000	3134,20	4150,286	-1016,09
72000	7064,47	9387,948	-2323,48
75000	7697,43	10028,453	-2331,02
78000	8346,53	10874,131	-2527,61
81000	9024,13	11719,621	-2695,49
84000	9745,93	12656,177	-2910,24
87000	10492,18	13857,85	-3365,67
90000	11289,23	14552,512	-3263,28
93000	12098,29	15564,01	-3465,72
96000	13013,87	16581,926	-3568,06
99000	13999,92	17679,527	-3679,60

Таблица 5.18 – Сравнение времени для метода с привязкой с одному ядру

Размер	<i>Ryzen 7 8845HS</i>	<i>Intel Core I7 12650H</i>	Разница
1000	1,53	1,61	-0,08
2000	6,04	6,15	-0,11
3000	13,22	11,567	1,65
4000	23,36	19,449	3,91
5000	36,23	30,255	5,97
18000	454,17	509,866	-55,70
21000	611,88	714,591	-102,71
24000	802,03	952,608	-150,57
27000	1008,08	1228,58	-220,50
30000	1246,88	1536,111	-289,23
33000	1502,35	1873,294	-370,95
36000	1787,42	2251,238	-463,82
39000	2089,89	2659,909	-570,02
42000	2426,12	3104,74	-678,63
45000	2784,41	3583,002	-798,59
48000	3165,17	4099,699	-934,53
72000	7284,12	9493,663	-2209,54
75000	7830,77	10263,768	-2433,00
78000	8444,09	11126,138	-2682,05

Продолжение таблицы 5.18

81000	9131,27	11998,486	-2867,22
84000	9853,77	12949,399	-3095,63
87000	10611,33	13895,052	-3283,72
90000	11420,74	14909,209	-3488,47
93000	12227,98	15895,953	-3667,98
96000	13152,88	16988,831	-3835,95
99000	14170,62	18087,707	-3917,09

Для начала рассмотрим поведение в однопоточном режиме. Как и следовало ожидать, включённость *Hyper-Threading* / *SMT* практически не влияет на однопоточный прогон: при запуске с привязкой к одному ядру логическая вторичная нить не используется, и результаты определяются прежде всего частотой конкретного физического ядра, локальностью данных в кэше и потерями на переключения. По предоставленным данным (таблицы 5.14, 5.15, 5.17, 5.18) можно сделать вывод, что при малых объёмах (порядка тысяч элементов) *Intel Core i7-12650H* демонстрирует небольшое преимущество. Однако с ростом объёма массива картина меняется: начиная с десятков тысяч элементов *Ryzen 7 8845HS* стабильно опережает *i7* по однопоточному времени, что указывает на лучшую устойчивость *Ryzen* при работе с большими объёмами данных и, вероятно, на более выгодную организацию кэша и поведение при интенсивных операциях обмена в памяти. В сумме: однопоточная эффективность зависит от размера задачи — *Intel* выигрывает на очень маленьких задачах, а *AMD* предпочтителен для средних и больших массивов.

Перейдём к многопоточному анализу (*OpenMP*). Результаты демонстрируют принципиальную разницу в масштабируемости и стабильности поведения двух платформ. Для большинства тестовых размеров при включённом *SMT* параллельные времена на *Ryzen* значительно ниже, чем на *Intel*. Это видно из таблицы 5.13, где значения времени *OpenMP* для *Ryzen* гораздо меньше, а разницы (*Ryzen* и *Intel*) отрицательны и велики по модулю (иногда на сотни и тысячи миллисекунд). Таблица ускорений показывает, что для *Ryzen* параллельная версия даёт крупное ускорение (порядка десятков раз для больших N значения *speedup* достигают 20), тогда как таблица ускорений для *i7* демонстрирует сильно фрагментированные и непостоянные значения: некоторые точки дают ускорение где-то в 7 раз, но множество точек — слабое или отсутствующее ускорение (и даже значения <1 в отдельных размерах). Эта вариативность подтверждает, что *AMD* обеспечивает лучшее и более предсказуемое масштабирование *OpenMP*-варианта пузырьковой сортировки.

Аппаратные счётчики (*perf*) дают ключ к интерпретации причин этих наблюдений. По таблицам метрик (5.5, 5.6 и 5.11, 5.12) видно, что при параллельном запуске *Intel* демонстрирует значительное увеличение числа

циклов и резкое падение эффективности исполнения (*insn per cycle*) в некоторых режимах: в ряде прогонов для *i7* при *OpenMP* наблюдается очень большое число *cycles* и большой рост *branch-miss / branch* события, что указывает на неэффективное распределение работы между ядрами или на дополнительные расходы, связанные с переключением между P/E-ядрами гибридной архитектуры. В то же время *Ryzen* показывает более компактные показатели по суммарным тактам для параллельной версии и более эффективность в этих запусках, что согласуется с лучшими параллельными временами и высоким *speedup*. При выключенном *SMT* ситуация смягчается: у *Intel* некоторые параллельные метрики становятся лучше, но в целом масштабируемость *i7* остаётся менее предсказуемой и более чувствительной к настройке аффинити и числу потоков, чем у *Ryzen*.

Эксперименты однозначно показывают, что архитектурные различия между *Intel Core i7-12650H* (гибридная P/E-архитектура) и *AMD Ryzen 7 8845HS* (традиционная многоядерная конфигурация) приводят к разным поведенческим паттернам. Для задач, где важна одиночная быстрая ветвь исполнения и очень маленькие объёмы — *i7* иногда даёт лучшее время; однако для реальных, масштабируемых задач сортировки и при полной загрузке всех потоков *Ryzen* в проведённых измерениях даёт более высокую и стабильную производительность. Множественные прогоны, использование *perf* и контроль аффинити позволила получить воспроизводимые данные и выявить именно те архитектурные особенности, которые лежат в основе наблюдаемых различий, что делает выводы надёжными и практически применимыми при подборе платформы под конкретную нагрузку.

ЗАКЛЮЧЕНИЕ

В рамках данного курсового проекта было проведено исследование производительности процессоров *Intel Core i7-12650H* и *AMD Ryzen 7 8845HS* при выполнении алгоритма пузырьковой сортировки. Основное внимание было уделено анализу вычислительной эффективности обоих процессоров в различных режимах работы — как в однопоточном, так и в многопоточном исполнении, а также при изменении числа активных ядер и потоков.

В процессе работы были подробно изучены технические характеристики выбранных процессоров, что позволило выявить ключевые архитектурные особенности, влияющие на производительность при выполнении вычислительных задач. На основе этих данных была разработана программная реализация алгоритма пузырьковой сортировки на языке C++, включающая возможность регулировки числа потоков и размера сортируемого массива. Это позволило оценить влияние архитектурных различий и многопоточности на скорость выполнения алгоритма.

Для повышения достоверности экспериментов использовались различные объёмы входных данных, что дало возможность исследовать масштабируемость и поведение процессоров при увеличении вычислительной нагрузки. Кроме того, проведён анализ полученных результатов, включающий сравнение времени выполнения сортировки, зависимость производительности от числа ядер и оценку эффективности использования вычислительных ресурсов.

Проведённые эксперименты показали, что при однопоточной загрузке различия в производительности между платформами носят размерно-зависимый характер: *Intel Core i7-12650H* демонстрирует небольшое преимущество на малых объёмах данных, тогда как при росте размера массива *AMD Ryzen 7 8845HS* устойчиво показывает лучшее однопоточное время выполнения, вероятно за счёт более эффективной организации кэша и поведения при интенсивной работе с памятью. В многопоточном режиме заметно преимущество *Ryzen* — параллельная реализация на этой платформе даёт существенное и стабильно воспроизводимое ускорение для больших *N* *speedup* достигает порядка десятков раз по отношению к последовательной версии.

Таким образом, результаты проведённого анализа подчёркивают важность выбора процессора в зависимости от характера выполняемых задач. Полученные данные позволяют сделать вывод о том, как архитектура и организация вычислительных ядер влияют на эффективность выполнения алгоритмов сортировки. Проведённая работа способствовала углублению понимания особенностей современных процессоров и их поведения в условиях вычислительных нагрузок, что может быть полезно для дальнейших исследований в области оптимизации алгоритмов и оценки производительности вычислительных систем.

СПИСОК ЛИТЕРАТУРНЫХ ИСТОЧНИКОВ

- [1] Сравнение процессоров [Электронный ресурс]. – Режим доступа: <https://technical.city/ru/cpu>. – Дата доступа: 26.09.2025.
- [2] Ryzen 5 5500U: технические характеристики и тесты [Электронный ресурс]. – Режим доступа: <https://technical.city/ru/cpu/Ryzen-5-5500U>. – Дата доступа: 26.09.2025.
- [3] Обзор процессора Intel Core i5-1135G7 [Электронный ресурс]. – Режим доступа: <https://notebookblog.ru/obzory/protssessory/protssessory-intel/obzor-i-testirovanie-protssessora-intel-core-i5-1135g7>. – Дата доступа: 26.09.2025.
- [4] Benchmark on Cinebench [Электронный ресурс]. – <https://www.xda-developers.com/how-benchmark-gpu-cinebench/>. – Дата доступа: 26.09.2023.
- [5] Geekbench 6 [Электронный ресурс]. – <https://www.geekbench.com/>. – Дата доступа: 26.09.2025.
- [6] CPU Benchmarks [Электронный ресурс]. – Режим доступа: <https://www.intel.com/gaming/resources/read-cpu-benchmarks/>. – Дата доступа: 26.09.2025.
- [7] Intel Core i5 1135G7 vs AMD Ryzen 5 5500U [Электронный ресурс]. – Режим доступа: <https://nanoreview.net/ru/cpu-compare/intel-core-i5-1135g7-vs-amd-ryzen-5-5500u>. – Дата доступа: 28.09.2025.
- [8] Операционная система Linux [Электронный ресурс]. – Режим доступа: <https://gitverse.ru/blog/articles/development/45-operacionnaya-sistema-linux-chto-eto-za-os-zachem-ona-nuzhna-i-kak-ej-polzovatsya>. – Дата доступа: 28.09.2025.
- [9] Ядро ОС *Linux* [Электронный ресурс]. – Режим доступа: <https://younglinux.info/bash/kernellinux>. – Дата доступа: 28.09.2025.
- [10] Архитектура ОС Linux [Электронный ресурс]. – Режим доступа: <http://opensource.rules.net/arhitektura-linux.html>. – Дата доступа: 28.09.2025.
- [11] Файловые системы в Linux: их структура и типы [Электронный ресурс]. – Режим доступа: <https://timeweb.com/ru/community/articles/struktura-i-tipy-faylovyh-sistem-v-linux>. – Дата доступа: 28.09.2025.
- [12] История операционной системы Linux [Электронный ресурс]. – Режим доступа: <https://ask42.us/course/linux-introduction/chapter/history/>. – Дата доступа: 28.09.2025.
- [13] Linux kernel version history [Электронный ресурс]. – Режим доступа: https://en.wikipedia.org/wiki/Linux_kernel_version_history#Releases_5.x.y. – Дата доступа: 28.09.2025.
- [14] 10 причин выбрать Linux *Ubuntu* [Электронный ресурс]. – Режим доступа: <https://otus.ru/nest/post/1600/>. – Дата доступа: 28.09.2024.
- [15] Директивы OpenMP [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/ru-ru/cpp/parallel/openmp/reference/openmp-directives?view=msvc-170>. – Дата доступа: 28.09.2025.

[16] Microsoft: Processes and threads [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/ru-ru/windows/win32/procthread/processes-and-threads>. – Дата доступа: 27.10.2025.

[17] CPU Affinity [Электронный ресурс]. – Режим доступа: <https://www.cryptoprofi.info/?p=6848>. – Дата доступа: 27.10.2025.

[18] Соловьев, В. П. Основы численных методов / В. П. Соловьев, Т. М. Кривоносова, В. Л. Смирнов. – Минск: БГУИР, 2011. – 131с.

[19] pthread – Linux manual page [Электронный ресурс]. – Режим доступа: https://man7.org/linux/man-pages/man3/pthread_setaffinity_np.3.html. – Дата доступа: 01.11.2025.

[20] Профилирование и трейсинг с perf [Электронный ресурс]. – Режим доступа: <https://habr.com/ru/companies/first/articles/442738/>. – Дата доступа: 01.11.2025.

ПРИЛОЖЕНИЕ А
(обязательное)
Справка о проверке на заимствования



Рисунок А.1 – Справка о проверке на заимствования

ПРИЛОЖЕНИЕ Б

(обязательное)

Листинг программного кода

```
main.cpp

#include <bits/stdc++.h>
#include <omp.h>
using namespace std;
using hr_clock = chrono::steady_clock;

enum Algorithm { SEQ_BUBBLE, PAR_ODD_EVEN, BOTH };

static void print_usage(const char* prog) {
    cout << "Usage: " << prog << " [options]\n"
        << "Options:\n"
        << "  --sizes s1,s2,...      Comma-separated array sizes (default:
1000,2000,5000,10000)\n"
        << "  --runs N                Number of runs per configuration
(default: 3)\n"
        << "  --max-threads T        Max number of OpenMP threads to test
(default: number of hardware threads)\n"
        << "  --seed S                RNG seed (default: time-based)\n"
        << "  --output file.csv       Output CSV file (default:
results.csv)\n"
        << "  --alg {seq|par|both}    Algorithm to test (default: both)\n"
        << "  --help                  Show this message\n";
}

static vector<int> gen_random_array(size_t n, mt19937 &rng, int minv = 0, int
maxv = 1000000) {
    vector<int> a(n);
    uniform_int_distribution<int> dist(minv, maxv);
    for (size_t i = 0; i < n; ++i) a[i] = dist(rng);
    return a;
}

static bool is_sorted_ascending(const vector<int>& a) {
    for (size_t i = 1; i < a.size(); ++i)
        if (a[i-1] > a[i]) return false;
    return true;
}

static void sequential_bubble_sort(vector<int>& a) {
    size_t n = a.size();
    if (n < 2) return;
    for (size_t pass = 0; pass < n - 1; ++pass) {
        bool swapped = false;
        for (size_t i = 0; i < n - 1 - pass; ++i) {
            if (a[i] > a[i+1]) {
                swap(a[i], a[i+1]);
                swapped = true;
            }
        }
        if (!swapped) break;
    }
}

static void parallel_odd_even_sort(vector<int>& a) {
    size_t n = a.size();
```

```

    if (n < 2) return;
    for (size_t phase = 0; phase < n; ++phase) {
        if (phase % 2 == 0) {
            // Чётная фаза (even phase)
            #pragma omp parallel for schedule(static)
            for (size_t i = 0; i + 1 < n; i += 2) {
                if (a[i] > a[i+1]) {
                    swap(a[i], a[i+1]);
                }
            }
        } else {
            // Нечётная фаза (odd phase)
            #pragma omp parallel for schedule(static)
            for (size_t i = 1; i + 1 < n; i += 2) {
                if (a[i] > a[i+1]) {
                    swap(a[i], a[i+1]);
                }
            }
        }
    }
}

static double duration_ms(const hr_clock::time_point& s, const
hr_clock::time_point& e) {
    return chrono::duration_cast<chrono::duration<double, milli>>(e -
s).count();
}

static vector<size_t> parse_sizes(const string &s) {
    vector<size_t> out;
    string token;
    stringstream ss(s);
    while (getline(ss, token, ',')) {
        try {
            size_t v = stoul(token);
            if (v > 0) out.push_back(v);
        } catch (...) { }
    }
    return out;
}

int main(int argc, char** argv) {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    // Параметры по умолчанию
    vector<size_t> sizes = {1000, 2000, 5000, 10000};
    int runs = 3;
    int max_threads = omp_get_max_threads();
    if (max_threads < 1) max_threads = 1;
    unsigned seed =
(unsigned)chrono::system_clock::now().time_since_epoch().count();
    string output_file = "results.csv";
    Algorithm alg = BOTH;

    // Парсинг аргументов
    for (int i = 1; i < argc; ++i) {
        string a = argv[i];
        if (a == "--help" || a == "-h") { print_usage(argv[0]); return 0; }
        else if (a == "--sizes" && i+1 < argc) {
            sizes = parse_sizes(argv[++i]);
            if (sizes.empty()) {

```

```

        cerr << "Invalid --sizes argument\n"; return 1;
    }
} else if (a == "--runs" && i+1 < argc) {
    runs = stoi(argv[++i]);
    if (runs <= 0) { cerr << "Invalid --runs\n"; return 1; }
} else if (a == "--max-threads" && i+1 < argc) {
    max_threads = stoi(argv[++i]);
    if (max_threads <= 0) max_threads = 1;
} else if (a == "--seed" && i+1 < argc) {
    seed = (unsigned)stoul(argv[++i]);
} else if (a == "--output" && i+1 < argc) {
    output_file = argv[++i];
} else if (a == "--alg" && i+1 < argc) {
    string v = argv[++i];
    if (v == "seq") alg = SEQ_BUBBLE;
    else if (v == "par") alg = PAR_ODD_EVEN;
    else if (v == "both") alg = BOTH;
    else { cerr << "Invalid --alg value\n"; return 1; }
} else {
    cerr << "Unknown argument: " << a << "\n";
    print_usage(argv[0]);
    return 1;
}
}

ofstream fout(output_file);
if (!fout) {
    cerr << "Cannot open output file: " << output_file << "\n";
    return 1;
}
fout << "algorithm,threads,size,avg_time_ms,sorted,seed\n";

mt19937 rng(seed);

for (size_t size : sizes) {
    // подготовка баз для всех прогонов
    vector<vector<int>> bases;
    bases.reserve(runs);
    for (int run = 0; run < runs; ++run) {
        bases.push_back(gen_random_array(size, rng));
    }

    // ----- Последовательная версия (усреднение по runs) -----
    if (alg == SEQ_BUBBLE || alg == BOTH) {
        double sum_ms = 0.0;
        bool all_sorted = true;
        for (int run = 0; run < runs; ++run) {
            vector<int> a = bases[run]; // копия
            auto t0 = hr_clock::now();
            sequential_bubble_sort(a);
            auto t1 = hr_clock::now();
            double ms = duration_ms(t0, t1);
            sum_ms += ms;
            bool sorted = is_sorted_ascending(a);
            if (!sorted) all_sorted = false;
            cout << "[seq] size=" << setw(6) << size << " run=" <<
(run+1) << " time=" << ms << " ms sorted=" << sorted << "\n";
        }
        double avg = sum_ms / runs;
        fout << "sequential,1," << size << "," << fixed <<
setprecision(3) << avg << "," << (all_sorted? "1":"0") << "," << seed <<
"\n";
    }
}

```

```

    }

    // ----- Параллельная версия (усреднение по runs) -----
    if (alg == PAR_ODD_EVEN || alg == BOTH) {
        for (int t = 1; t <= max_threads; ++t) {
            // Устанавливаем число потоков OpenMP
            omp_set_num_threads(t);

            double sum_ms = 0.0;
            bool all_sorted = true;
            for (int run = 0; run < runs; ++run) {
                vector<int> a = bases[run]; // копия
                auto t0 = hr_clock::now();
                parallel_odd_even_sort(a);
                auto t1 = hr_clock::now();
                double ms = duration_ms(t0, t1);
                sum_ms += ms;
                bool sorted = is_sorted_ascending(a);
                if (!sorted) all_sorted = false;
                cout << "[par] thr=" << setw(2) << t << " size=" <<
setw(6) << size << " run=" << (run+1) << " time=" << ms << " ms sorted=" <<
sorted << "\n";
            }
            double avg = sum_ms / runs;
            fout << "parallel_odd_even," << t << "," << size << "," <<
fixed << setprecision(3) << avg << "," << (all_sorted? "1":"0") << "," <<
seed << "\n";
        }
    }

    cout.flush();
}

fout.close();
cout << "\nCompleted. Averaged results written to " << output_file <<
"\n";
cout << "CSV columns: algorithm,threads,size,avg_time_ms,sorted,seed\n";
cout << "Note: each avg_time_ms is arithmetic mean over " << runs << "
runs (with different random inputs), and for a given run the same input was
used for all tested thread counts.\n";
return 0;
}

```

ПРИЛОЖЕНИЕ В
(обязательное)
Функциональная схема алгоритма
реализующего программное средство

ПРИЛОЖЕНИЕ Г
(обязательное)
Блок схема алгоритма,
реализующего программное средство

ПРИЛОЖЕНИЕ Д
(обязательное)
Графики сравнений производительности процессоров

ПРИЛОЖЕНИЕ Е
(обязательное)
Графический интерфейс пользователя

ПРИЛОЖЕНИЕ Ж
(обязательное)
Ведомость курсового проекта